Submitted By:

Latif Waqas– 2025-CYS-33 Assignment:

Pacman Game Project

Submitted To:

Sir Shahmeer Nawaz

For the fulfillment of,

Object-Oriented Programming Lab Department of Computer Engineering

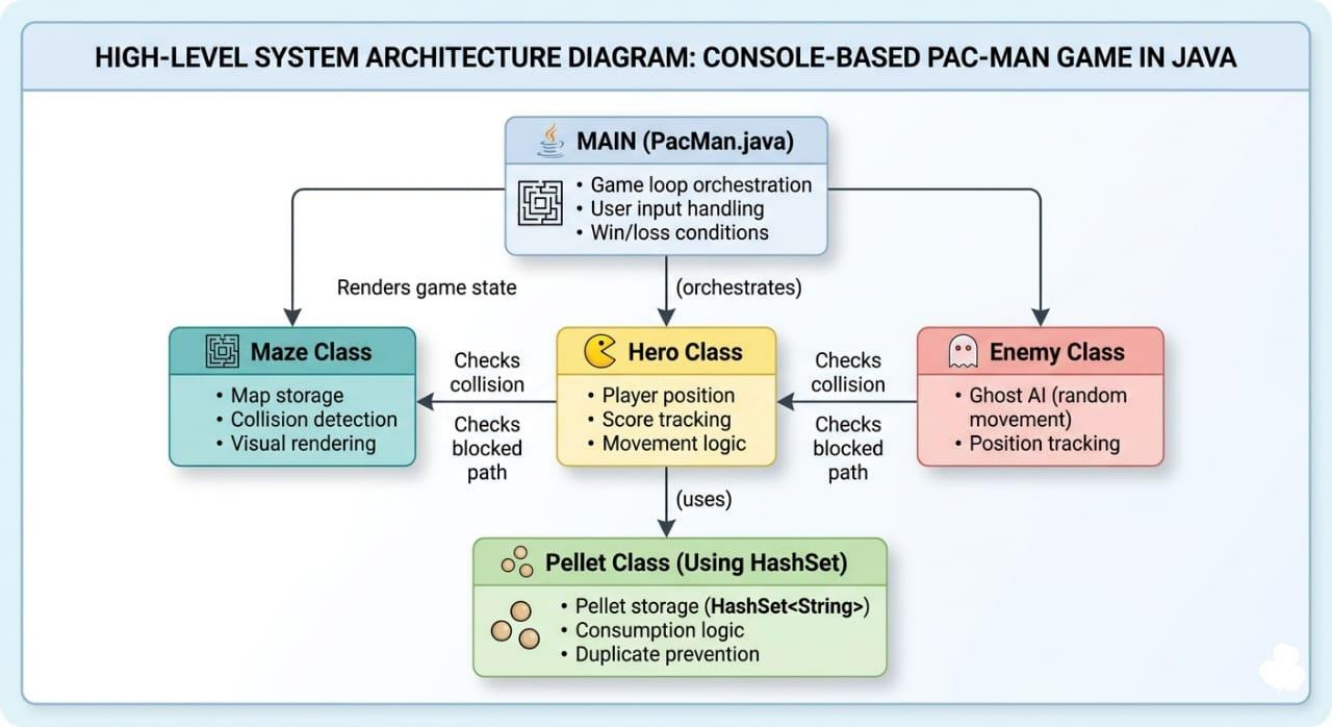
University of Engineering and Technology, Lahore

1. Abstract

This report presents the design and implementation of a simplified, text-based Pac-Man game using Java. The game features a hero character who navigates a maze, collects pellets, and avoids an enemy ghost. The primary focus is on demonstrating core ObjectOriented Programming (OOP) principles including encapsulation, modularity, and the effective use of Java Collections. The application runs in a console environment and provides real-time keyboard input for player movement.

2. System Architecture

The system follows a modular architecture with five primary classes, each handling a distinct responsibility. The architecture is eventdriven, where user input triggers movement, collision detection, and game state updates.



2.1 Class Responsibilities Table

Class Responsibility Key Methods

PacMan (Main) : Game loop orchestration, user input handling, win/loss conditions main()

Maze Map storage, collision detection, visual rendering setup(), render(), blocked()

Hero Player position, score tracking, movement logic movePlayer(), addPoint()

Enemy Ghost AI : (random movement), position tracking randomMove()

Pellet Pellet storage using HashSet, consumption logic place(), consume(), exists()

3. Detailed Class Design

3.1 PacMan (Main Driver)

This class contains the game loop, which continues until either the player collides with the ghost or all pellets are collected.

Key Logic Flow:

```java while
(true) {
   maze.render(player, ghost, pellets);  // Display game state
char key = input.next().charAt(0);    // Get user input
player.movePlayer(key, maze);         // Update player position
pellets.consume(...);                 // Check for pellet collection
ghost.randomMove(maze);               // Move enemy
   // Collision & win checks
}
```

Design Choice: The main class acts solely as an orchestrator, delegating all domain logic to specialized classes.

3.2 Maze Class

The maze is stored as a 10×10 2D character array. The layout is hardcoded using a string array for simplicity and reproducibility.

Collision Detection:

```java
public boolean blocked(int x, int y) {
   if (x < 0 || y < 0 || x >= r || y >= c) return true;
return grid[x][y] == '#';
}
```

Rendering Logic:

· C → Hero (Pac-Man)
· X → Ghost (Enemy)
· # → Wall
· . → Pellet (disappears when consumed)
· (space) → Empty path

3.3 Hero Class:

The hero maintains its coordinates and score. Movement is validated against maze walls before updating position.

State Variables:

Variable Type Description
row int Current row index col
int Current column index
points int Number of pellets eaten

3.4 Enemy Class

The enemy implements simple random movement AI. At each turn, it randomly selects one of four directions (up, down, left, right). If the chosen direction leads to a wall, the ghost stays in place for that turn.

Movement Directions:

```java
int[][] moves = {{0,1}, {0,-1}, {1,0}, {-1,0}};
// Right, Left, Down, Up
```

3.5 Pellet Class (Using HashSet)

This class demonstrates efficient use of Java Collections. Pellets are stored as string keys in a HashSet<String>.

Why HashSet?

· O(1) average time complexity for add, remove, and contains operations
· Automatic duplicate prevention
· Simple key generation: row + "-" + col

Key Methods:

```java
public void place(int x, int y) { items.add(x + "-" + y); }
public boolean consume(int x, int y) { return items.remove(x + "-" + y); }
public boolean exists(int x, int y) { return items.contains(x + "-" + y); }
public boolean finished() { return items.isEmpty(); }
```

4. Data Structures Used

Data Structure Purpose Complexity char[][]
Maze grid storage O(1) access
HashSet<String> Pellet collection O(1) avg. for add/remove/contains
Scanner User input handling N/A
Random Ghost movement randomness O(1)

---

5. Sample Output

```text
##########
#C.......#
#.####...#
#.#..#.#.#
#.#..#.#.#
#.####.#.#
#.......#
#.######.#
#......X#
##########
Points: 3
```

Move (W/A/S/D): s
```

---

## 6. Limitations and Future Improvements

### Current Limitations

1. Ghost AI: Random movement is not intelligent; ghosts do not chase the player.
2. Fixed Maze Size: Maze dimensions are hardcoded (10×10).
3. Console-Based: No graphical interface or sound effects.
4. Single Enemy: Only one ghost exists in the game.

### Proposed Enhancements

Enhancement Description Difficulty
Chase AI Implement BFS pathfinding so ghost follows player Medium
Multiple Enemies Create an enemy list with independent movement Easy
Power Pellets Add special pellets that allow eating ghosts Medium
Levels Increase difficulty by speeding up ghosts Easy
GUI Version Migrate to Swing/JavaFX for visual interface Hard
High Score Save top scores to a file Medium
```

---

7. Compilation and Execution Instructions

Prerequisites

· Java Development Kit (JDK) 8 or higher

Steps

1. Save all code in a single file named PacMan.java 2. Open terminal/command prompt in the file directory
3. Compile:
```bash
javac PacMan.java
```
4. Run:
```bash
java PacMan
```

Controls

Key Action
W Move Up
A Move Left
S Move Down
D Move Right

---

8. Conclusion

This project successfully implements a functional Pac-Man style game using core Java and Object-Oriented Programming principles. The architecture separates concerns effectively, making the code maintainable and extensible. The use of HashSet for pellet management demonstrates efficient data structure selection. While the current implementation is console-based with simple random ghost movement, it provides a solid foundation for future enhancements such as intelligent ghost AI, multiple levels, and graphical interfaces.